HPI Hasso Plattner Institut
Digital Engineering · Universität Potsdam

Programmiertechnik II

Leonhard Hennicke

Tutorium 2: Input Streams, Shunting-Yard-Algorithmus

Überblick

1. Floats aus std::cin lesen
2. Rule of Three
3. Shunting-Yard Algorithmus

Floats aus std::cin lesen

- std::cin >> x liest direkt in Variable, durch Lerzeichen begrenzt
  - Ungeeignet bei verschiedenen Datentypen in einer Zeile
- std::getline(std::cin, line) liest die gesamte Zeile in einen std::string
  - Um mehrstelliges Token (z. B. Floats) zu lesen , iterieren wir so lange, wie das aktuelle Zeichen zum Token gehört (z.B. Ziffer oder "." bei Floats)
- std::stof(token) konvertiert den gesammelten String in einen float
  - Kann std::invalid_argument werfen → try/catch

token_parsing.cpp

Rule of Three (Shallow Copy Problem)

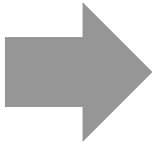
- Verwaltet eine Klasse Heap-Speicher wird per default Copy-Konstruktor nur der Zeiger kopiert, nicht die Daten
- Zwei Objekte teilen sich eine Adresse → werden immer gemeinsam verändert
- Wenn beide Destruktoren laufen, wird delete zweimal auf dieselbe Adresse aufgerufen → **undefiniertes Verhalten / Absturz**
- **Die Lösung — tiefe Kopie (Deep Copy) für alle drei:**
  - **Copy-Konstruktor**: *neuen* Speicher allozieren und den *Inhalt* kopieren (*value(new int(*other.value))*)
  - **Copy-Zuweisungsoperator**: vor dem Kopieren auf Selbstzuweisung prüfen (*if (this == &other) return *this;*) — ohne diesen Schutz würde "*a = a*" evtl. den Speicher freigeben und danach darauf zugreifen
  - **Destruktor**: allozierten Speicher freigeben

rule_of_three.cpp

Shunting-Yard Algorithmus

Infix-Notation

(2.0 * 3.0 + (2.0 / 4.0)) * 7.0



Postfix-Notation

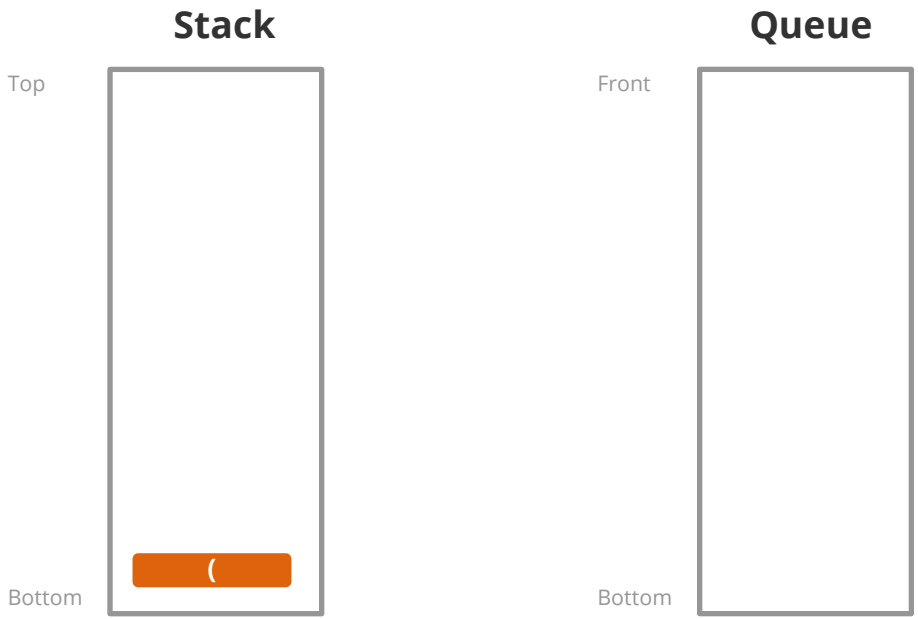
2.0 3.0 * 2.0 4.0 / + 7.0 *

Shunting-Yard Algorithmus

```
Eingabe: string von Tokens, Gleitkommazahlen separiert durch " "
S = Stack
Q = Queue
while tokens not empty:
    t = nächster token
    if t ist Gleitkommazahl:
        push(Q, t)
    else if t ist Operator:
        while not is_empty(S) and
            top(S) ist Operator and
            t hat Präzedenz kleiner gleich top(S):
                push(Q, pop(S)) # Stack-Top zur Ausgabe
        push(S,t)
    else if t ist "(":
        push(S,t)
    else if t ist ")":
        while top(S) ist nicht "(":
            push(Q, pop(S))
        pop(S)
while not is_empty(S)
    if top(S) ist "(": Fehler
    push(Q,pop(s))
return Q (als string)
```

Shunting-Yard Algorithmus

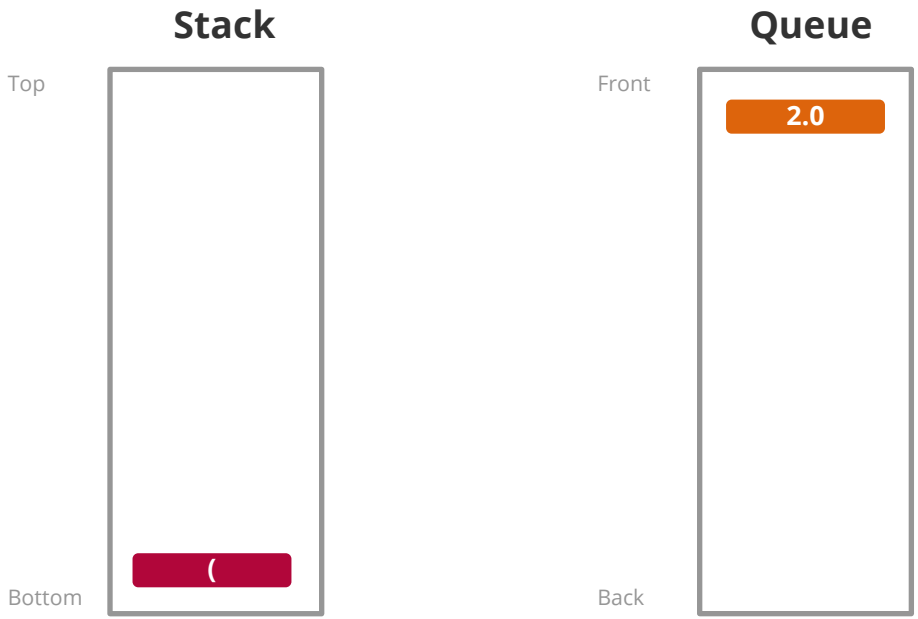
Input: (2.0 * 3.0 + (2.0 / 4.0)) * 7.0




```
while tokens not empty:
    t = nächster token
    if t ist Gleitkommazahl:
        push(Q, t)
    else if t ist Operator:
        while not is_empty(S) and
            top(S) ist Operator and
            t hat Präzedenz kleiner gleich top(S):
                push(Q, pop(S)) # Stack-Top zur Aus
        push(S,t)
    else if t ist "(":
        push(S,t)
    else if t ist ")":
        while top(S) ist nicht "(":
            push(Q, pop(S))
        pop(S)
while not is_empty(S)
    if top(S) ist "(": Fehler
    push(Q,pop(s))
return Q (als string)
```

Shunting-Yard Algorithmus

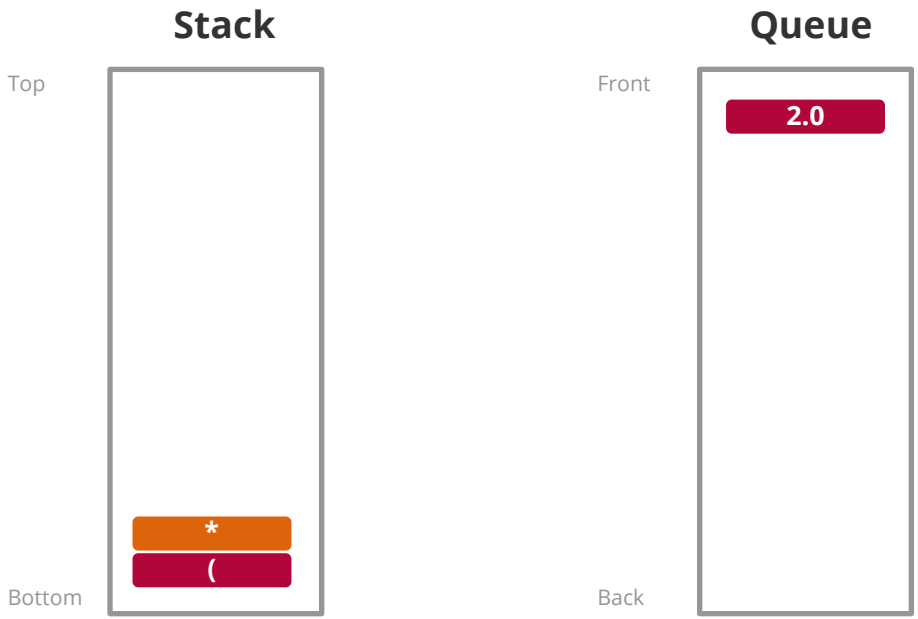
Input: (2.0 * 3.0 + (2.0 / 4.0)) * 7.0




```
while tokens not empty:
    t = nächster token
    if t ist Gleitkommazahl:
        push(Q, t)
    else if t ist Operator:
        while not is_empty(S) and
            top(S) ist Operator and
            t hat Präzedenz kleiner gleich top(S):
                push(Q, pop(S)) # Stack-Top zur Aus
        push(S,t)
    else if t ist "(":
        push(S,t)
    else if t ist ")":
        while top(S) ist nicht "(":
            push(Q, pop(S))
        pop(S)
while not is_empty(S)
    if top(S) ist "(": Fehler
    push(Q,pop(s))
return Q (als string)
```

Shunting-Yard Algorithmus

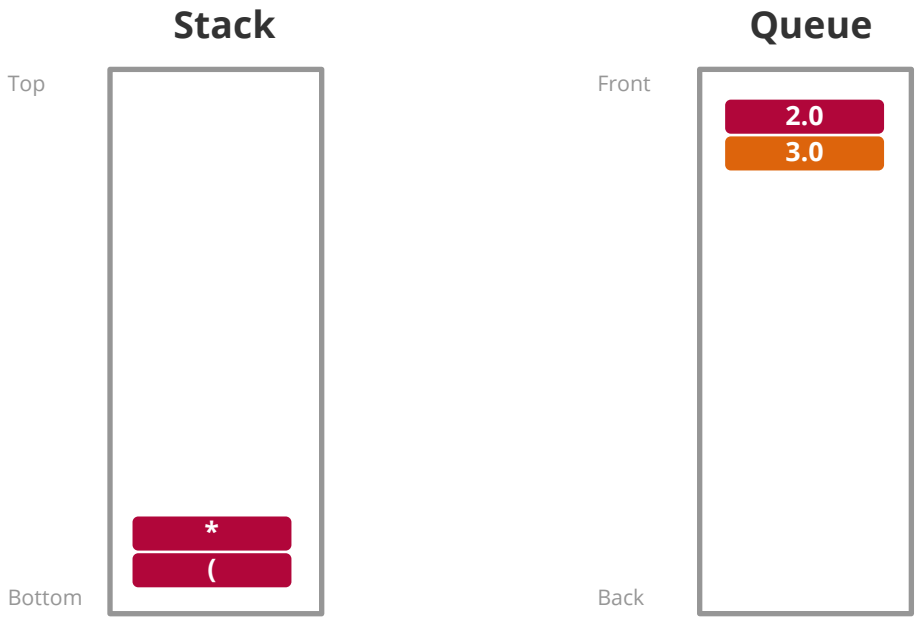
Input: (2.0 * 3.0 + (2.0 / 4.0)) * 7.0




```
while tokens not empty:
    t = nächster token
    if t ist Gleitkommazahl:
        push(Q, t)
    else if t ist Operator:
        while not is_empty(S) and
            top(S) ist Operator and
            t hat Präzedenz kleiner gleich top(S):
                push(Q, pop(S)) # Stack-Top zur Aus
        push(S,t)
    else if t ist "(":
        push(S,t)
    else if t ist ")":
        while top(S) ist nicht "(":
            push(Q, pop(S))
        pop(S)
while not is_empty(S)
    if top(S) ist "(": Fehler
    push(Q,pop(s))
return Q (als string)
```

Shunting-Yard Algorithmus

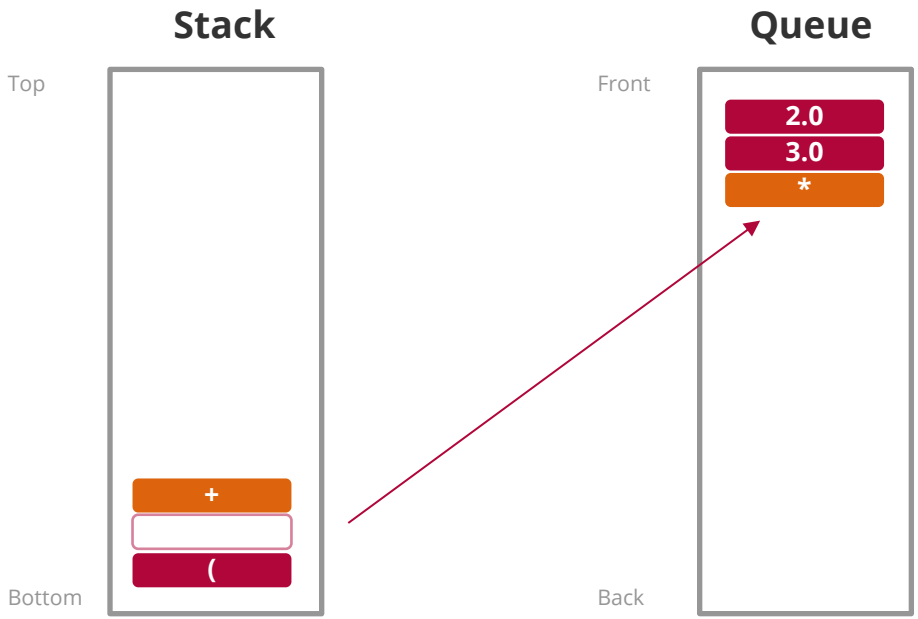
Input: (2.0 * 3.0 + (2.0 / 4.0)) * 7.0




```
while tokens not empty:
    t = nächster token
    if t ist Gleitkommazahl:
        push(Q, t)
    else if t ist Operator:
        while not is_empty(S) and
            top(S) ist Operator and
            t hat Präzedenz kleiner gleich top(S):
                push(Q, pop(S)) # Stack-Top zur Aus
        push(S,t)
    else if t ist "(":
        push(S,t)
    else if t ist ")":
        while top(S) ist nicht "(":
            push(Q, pop(S))
        pop(S)
while not is_empty(S)
    if top(S) ist "(": Fehler
    push(Q,pop(s))
return Q (als string)
```

Shunting-Yard Algorithmus

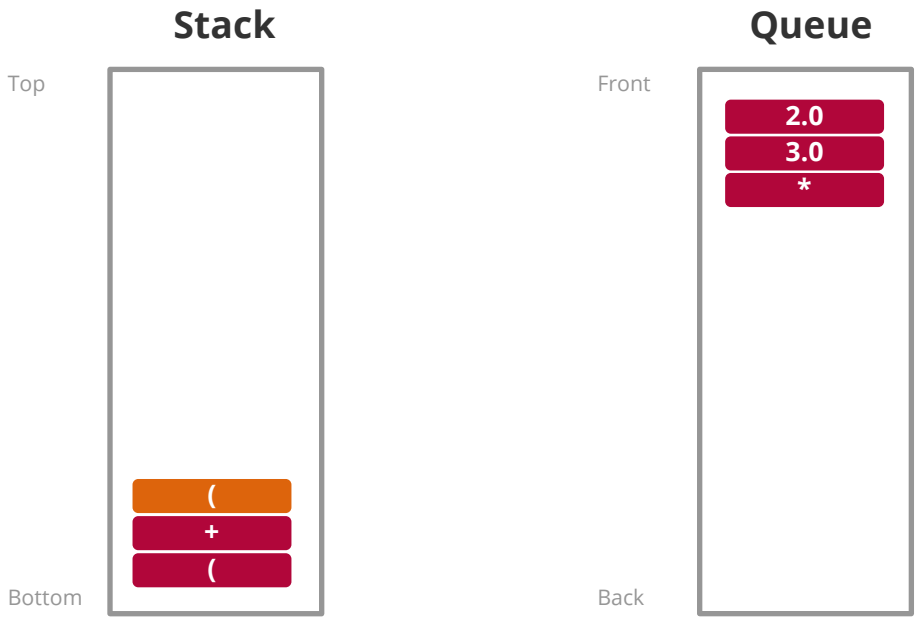
Input: (2.0 * 3.0 + (2.0 / 4.0)) * 7.0




```
while tokens not empty:
    t = nächster token
    if t ist Gleitkommazahl:
        push(Q, t)
    else if t ist Operator:
        while not is_empty(S) and
            top(S) ist Operator and
            t hat Präzedenz kleiner gleich top(S):
                push(Q, pop(S)) # Stack-Top zur Aus
        push(S,t)
    else if t ist "(":
        push(S,t)
    else if t ist ")":
        while top(S) ist nicht "(":
            push(Q, pop(S))
        pop(S)
while not is_empty(S)
    if top(S) ist "(": Fehler
    push(Q,pop(s))
return Q (als string)
```

Shunting-Yard Algorithmus

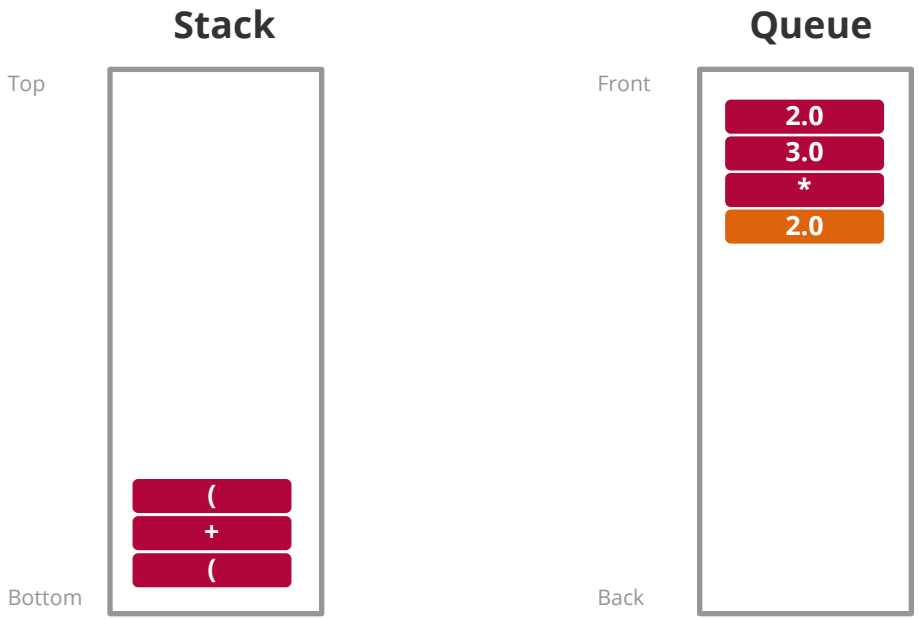
Input: (2.0 * 3.0 + (2.0 / 4.0)) * 7.0




```
while tokens not empty:
    t = nächster token
    if t ist Gleitkommazahl:
        push(Q, t)
    else if t ist Operator:
        while not is_empty(S) and
            top(S) ist Operator and
            t hat Präzedenz kleiner gleich top(S):
                push(Q, pop(S)) # Stack-Top zur Aus
        push(S,t)
    else if t ist "(":
        push(S,t)
    else if t ist ")":
        while top(S) ist nicht "(":
            push(Q, pop(S))
        pop(S)
while not is_empty(S)
    if top(S) ist "(": Fehler
    push(Q,pop(s))
return Q (als string)
```

Shunting-Yard Algorithmus

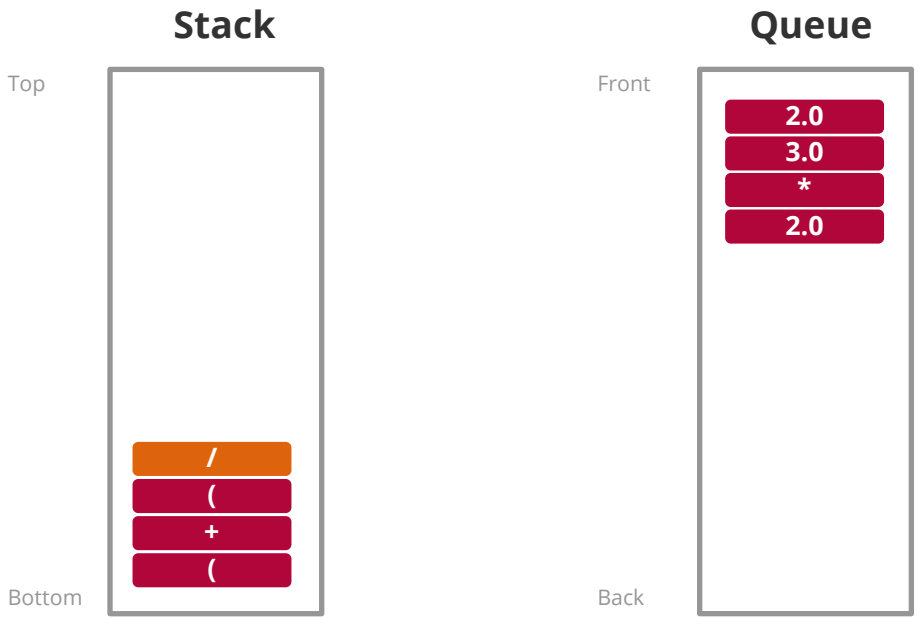
Input: (2.0 * 3.0 + (2.0 / 4.0)) * 7.0




```
while tokens not empty:
    t = nächster token
    if t ist Gleitkommazahl:
        push(Q, t)
    else if t ist Operator:
        while not is_empty(S) and
            top(S) ist Operator and
            t hat Präzedenz kleiner gleich top(S):
                push(Q, pop(S)) # Stack-Top zur Aus
        push(S,t)
    else if t ist "(":
        push(S,t)
    else if t ist ")":
        while top(S) ist nicht "(":
            push(Q, pop(S))
        pop(S)
while not is_empty(S)
    if top(S) ist "(": Fehler
    push(Q,pop(s))
return Q (als string)
```

Shunting-Yard Algorithmus

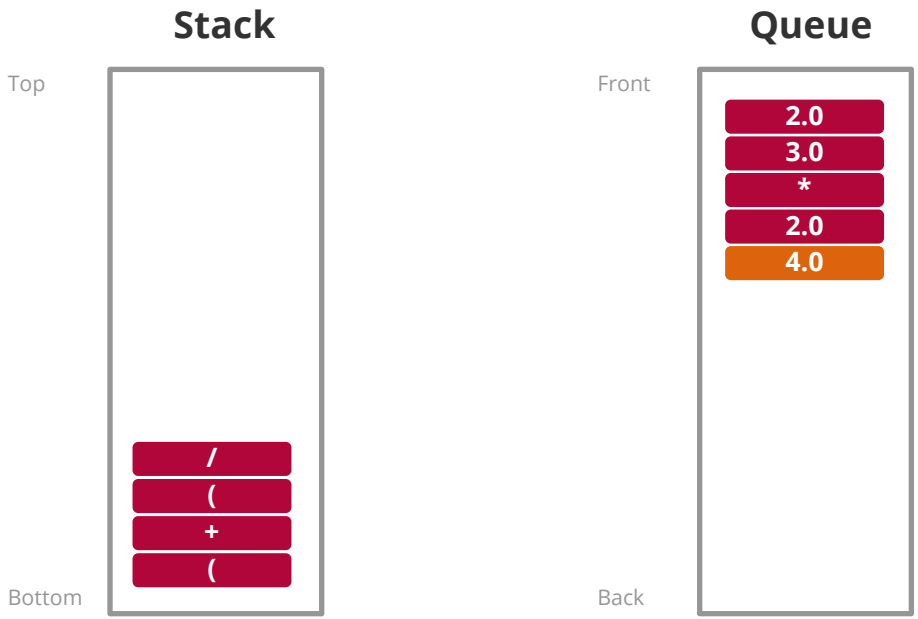
Input: (2.0 * 3.0 + (2.0 / 4.0)) * 7.0




```
while tokens not empty:
    t = nächster token
    if t ist Gleitkommazahl:
        push(Q, t)
    else if t ist Operator:
        while not is_empty(S) and
            top(S) ist Operator and
            t hat Präzedenz kleiner gleich top(S):
                push(Q, pop(S)) # Stack-Top zur Aus
        push(S,t)
    else if t ist "(":
        push(S,t)
    else if t ist ")":
        while top(S) ist nicht "(":
            push(Q, pop(S))
        pop(S)
while not is_empty(S)
    if top(S) ist "(": Fehler
    push(Q,pop(s))
return Q (als string)
```

Shunting-Yard Algorithmus

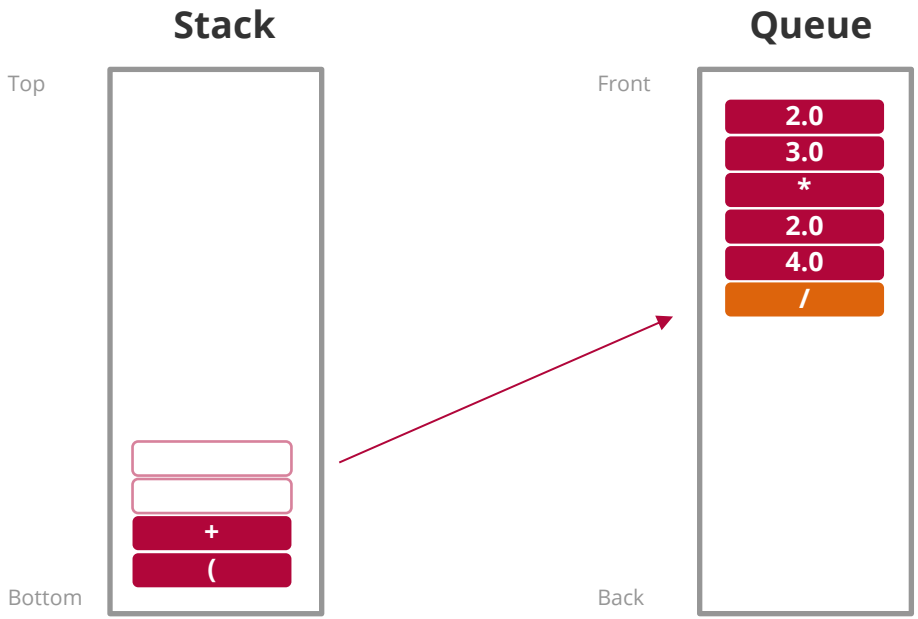
Input: (2.0 * 3.0 + (2.0 / 4.0)) * 7.0




```
while tokens not empty:
    t = nächster token
    if t ist Gleitkommazahl:
        push(Q, t)
    else if t ist Operator:
        while not is_empty(S) and
            top(S) ist Operator and
            t hat Präzedenz kleiner gleich top(S):
                push(Q, pop(S)) # Stack-Top zur Aus
        push(S,t)
    else if t ist "(":
        push(S,t)
    else if t ist ")":
        while top(S) ist nicht "(":
            push(Q, pop(S))
        pop(S)
while not is_empty(S)
    if top(S) ist "(": Fehler
    push(Q,pop(s))
return Q (als string)
```

Shunting-Yard Algorithmus

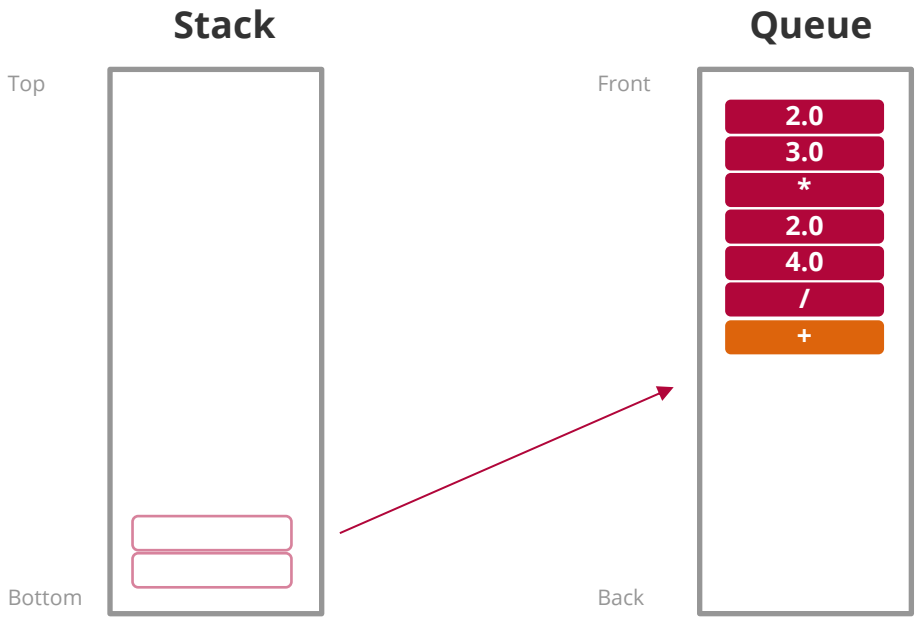
Input: (2.0 * 3.0 + (2.0 / 4.0)) * 7.0




```
while tokens not empty:
    t = nächster token
    if t ist Gleitkommazahl:
        push(Q, t)
    else if t ist Operator:
        while not is_empty(S) and
            top(S) ist Operator and
            t hat Präzedenz kleiner gleich top(S):
                push(Q, pop(S)) # Stack-Top zur Aus
        push(S,t)
    else if t ist "(":
        push(S,t)
    else if t ist ")":
        while top(S) ist nicht "(":
            push(Q, pop(S))
        pop(S)
while not is_empty(S)
    if top(S) ist "(": Fehler
    push(Q,pop(s))
return Q (als string)
```

Shunting-Yard Algorithmus

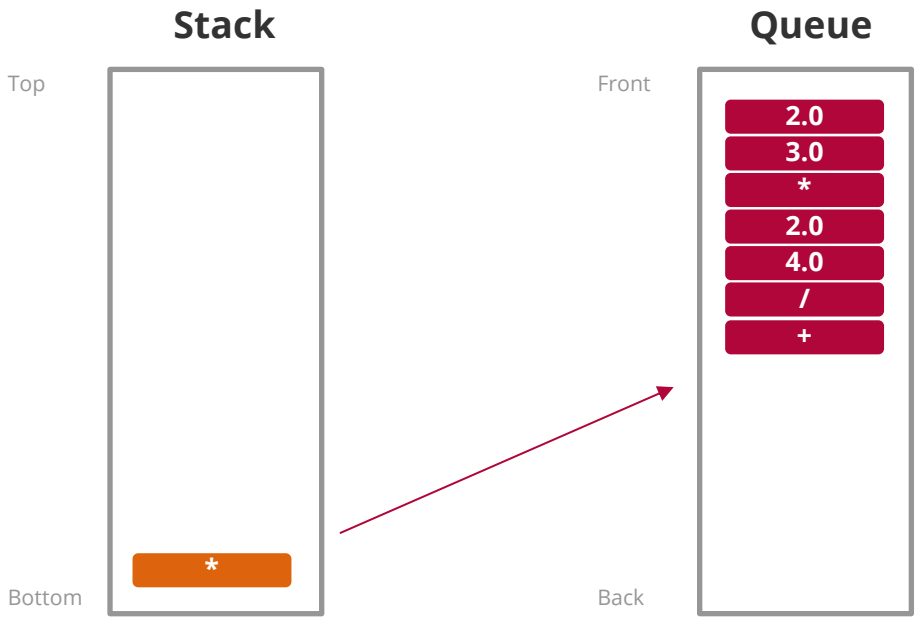
Input: (2.0 * 3.0 + (2.0 / 4.0)) * 7.0




```
while tokens not empty:
    t = nächster token
    if t ist Gleitkommazahl:
        push(Q, t)
    else if t ist Operator:
        while not is_empty(S) and
            top(S) ist Operator and
            t hat Präzedenz kleiner gleich top(S):
                push(Q, pop(S)) # Stack-Top zur Aus
        push(S,t)
    else if t ist "(":
        push(S,t)
    else if t ist ")":
        while top(S) ist nicht "(":
            push(Q, pop(S))
        pop(S)
while not is_empty(S)
    if top(S) ist "(": Fehler
    push(Q,pop(s))
return Q (als string)
```

Shunting-Yard Algorithmus

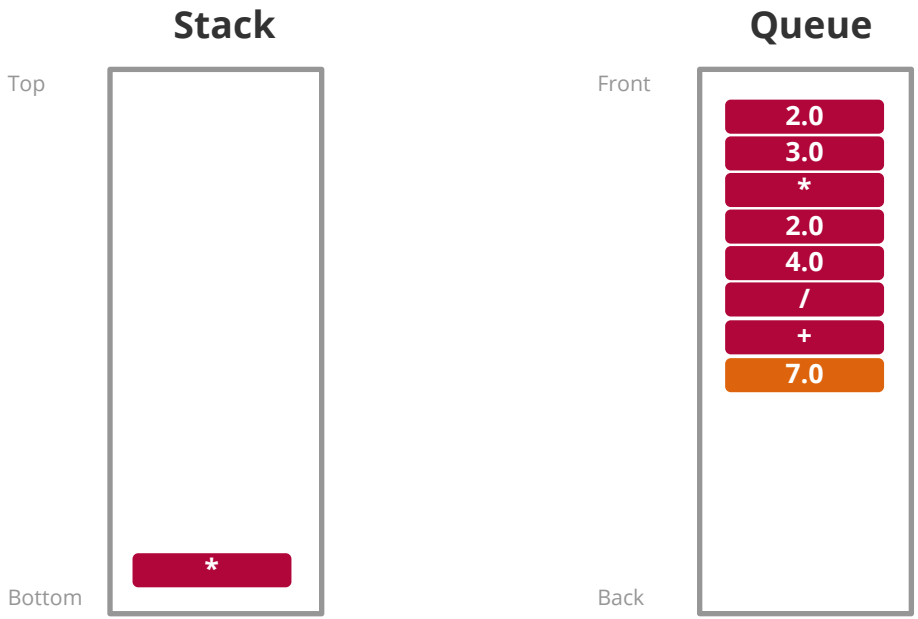
Input: (2.0 * 3.0 + (2.0 / 4.0)) * 7.0




```
while tokens not empty:
    t = nächster token
    if t ist Gleitkommazahl:
        push(Q, t)
    else if t ist Operator:
        while not is_empty(S) and
            top(S) ist Operator and
            t hat Präzedenz kleiner gleich top(S):
                push(Q, pop(S)) # Stack-Top zur Aus
        push(S,t)
    else if t ist "(":
        push(S,t)
    else if t ist ")":
        while top(S) ist nicht "(":
            push(Q, pop(S))
        pop(S)
while not is_empty(S)
    if top(S) ist "(": Fehler
    push(Q,pop(s))
return Q (als string)
```

Shunting-Yard Algorithmus

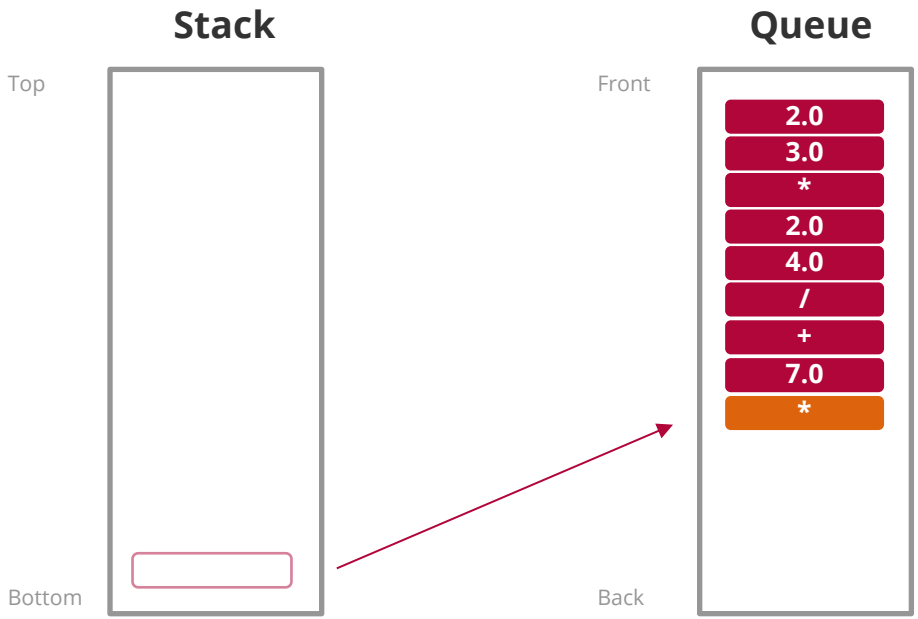
Input: (2.0 * 3.0 + (2.0 / 4.0)) * 7.0




```
while tokens not empty:
    t = nächster token
    if t ist Gleitkommazahl:
        push(Q, t)
    else if t ist Operator:
        while not is_empty(S) and
            top(S) ist Operator and
            t hat Präzedenz kleiner gleich top(S):
                push(Q, pop(S)) # Stack-Top zur Aus
        push(S,t)
    else if t ist "(":
        push(S,t)
    else if t ist ")":
        while top(S) ist nicht "(":
            push(Q, pop(S))
        pop(S)
while not is_empty(S)
    if top(S) ist "(": Fehler
    push(Q,pop(s))
return Q (als string)
```

Shunting-Yard Algorithmus

Input: (2.0 * 3.0 + (2.0 / 4.0)) * 7.0 _

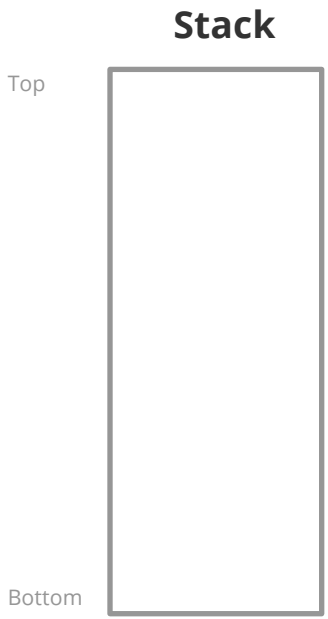


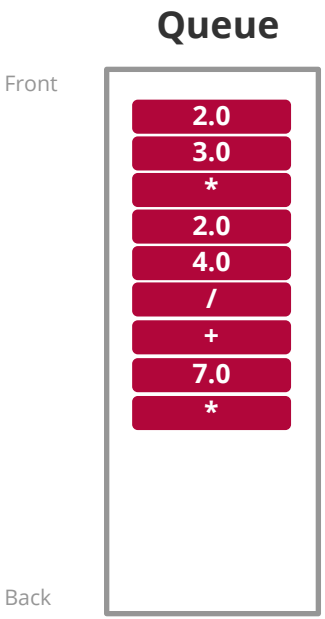

```
while tokens not empty:
    t = nächster token
    if t ist Gleitkommazahl:
        push(Q, t)
    else if t ist Operator:
        while not is_empty(S) and
            top(S) ist Operator and
            t hat Präzedenz kleiner gleich top(S):
                push(Q, pop(S)) # Stack-Top zur Aus
        push(S,t)
    else if t ist "(":
        push(S,t)
    else if t ist ")":
        while top(S) ist nicht "(":
            push(Q, pop(S))
        pop(S)
while not is_empty(S)
    if top(S) ist "(": Fehler
    push(Q,pop(s))
return Q (als string)
```

Shunting-Yard Algorithmus

Input: (2.0 * 3.0 + (2.0 / 4.0)) * 7.0

Output: 2.0 3.0 * 2.0 4.0 / + 7.0 *






```
while tokens not empty:
    t = nächster token
    if t ist Gleitkommazahl:
        push(Q, t)
    else if t ist Operator:
        while not is_empty(S) and
            top(S) ist Operator and
            t hat Präzedenz kleiner gleich top(S):
                push(Q, pop(S)) # Stack-Top zur Aus
        push(S,t)
    else if t ist "(":
        push(S,t)
    else if t ist ")":
        while top(S) ist nicht "(":
            push(Q, pop(S))
        pop(S)
while not is_empty(S)
    if top(S) ist "(": Fehler
    push(Q,pop(s))
return Q (als string)
```

Hasso Plattner Institut
Digital Engineering · Universität Potsdam

Viel Spaß, bis zur nächsten Vorlesung!